

Graph Neural Network for Quantum Annealing Embedding Results Prediction and Generation

Dimitri Vinci ID 223960 *dimitri.vinci@studenti.unitn.it*

I. INTRODUCTION

In the following document research project for the course Advanced Topics in Machine Learning and Optimization - a.y. 2023/2024 is presented. In the first section core concepts and motivation are exposed, following by two section describing experiments and work done. Two different (but connected) experiments are exposed: first experiment regard performance prediction of quantum annealing on a maximum independent set (MIS) task using quantum annealer minor embedding information via Graph Neural Network (GNN) compared to classical machine learning algorithm, while second experiment regard construction of a generative GNN model capable of automatic improvement of quantum annealer minor embedding on same MIS task. Given limited literature and previous work (as will be exposed thereafter) for both experiments, it was necessary both constructing a dataset and developing new graph neural network based model. Results seem indicate how GNN improve over classical algorithm over embedding prediction, while a generative neural network approach seem beneficial to improve performances confirming the hypothesis.

II. CORE CONCEPTS AND MOTIVATION

Here are exposed core concepts treated in the work as long as background and proposal.

A. Quantum Annealing

With end of Moore's law an increasing concern about limits of CPU, GPU and GPUs based supercomputer has raised [1][2]. Of all possible computational paradigms, quantum computation is one of the main approach that is thought be able to overcome classical CPU and GPU computation. There are great expectation about quantum computation [3] and quantum supremacy (meaning capability of quantum computation to solve classical untreatable problems) has been experimentally demonstrated for highly theoretical model [4]. Among the quantum computing models, adiabatic quantum computers (originally proposed by [5] and shown polinomially equivalent to quantum gate model [6][7]) are currently more readily available at user sites due to recent advancements in commercializing programmable quantum annealers by D-Wave Systems [8] (that is the quantum computing platform considered in this project). Quantum annealing is an annealing meta-heuristic (family of Metropolis/Hastings algorithms [9]) that employs adjustable quantum fluctuations into a problem. Quantum annealing answers to a typical problem of classical

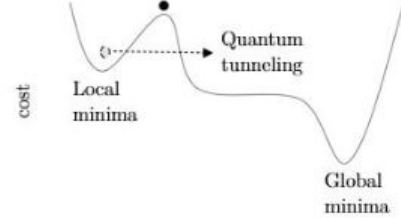


Fig. 1. Landscape of energy minimization problem; picture inspired by [11]

Thermal annealing (simulated annealing or classical annealing) meaning the landscape of the given Hamiltonian $\mathcal{H}_F$ (that represent the anneal problem at hand) could expose energy barriers around local minimums that can trap the system for a very long time. Quantum annealing introduces a transverse field Hamiltonian (also called the disordering Hamiltonian) $\mathcal{H}_D$ that does not commute with $\mathcal{H}_F$. Scaling it with gamma, creates a new time-dependent Hamiltonian,

$$\mathcal{H}(t) = \mathcal{H}_F + \Gamma^\top(t)\mathcal{H}_D$$

The disordering Hamiltonian $\mathcal{H}_D$ introduces kinetic energy to the annealing process in the form of quantum fluctuations (changing in the amount of energy at a point in space for extremely short time lapses as a result of the uncertainty principle [10]) of the solution space. The tunnel effect/quantum tunneling controlled by gamma (transverse field) allows to traverse through the solid interstate potential barriers (in this case of the problem landscape) by quantum jumps [9]. Quantum annealers are then a type of adiabatic quantum computer that provides a hardware implementation for finding the minimum energy configuration of Hamiltonians whose ground states represent optimum solutions of the original problems of interest.

B. D-Wave platform

Founded in 1999 by Haig Farris, Geordie Rose, Bob Wiens and Alexandre Zagoukin as a branch of University of British Columbia, With headquarters near Vancouver, Canada [12]. D-Wave is a full-stack technology provider in which annealer is provided along with other services [13]. A D-Wave platform comprises two main components: A processor chip that solves Ising Model by physical realization of a quantum annealing algorithm (QPU/IPU) and a front end server. The conventional (Intel) front end server is connected to the chip via control lines and an I/O subsystem. The front end receives instructions from the user and is accessed using a cloud computing model called Leap. The front end sends a problem instance to the

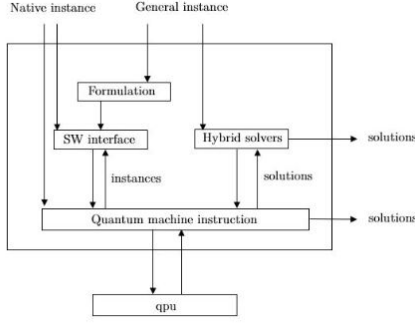


Fig. 2. Depiction of D-Wave system; picture inspired by [74]

hardware and receives a set of solutions in reply. Leap is used to submit problems providing access (via a problem API token) to a portfolio of quantum processing units (QPUs), hybrid solvers, classical solvers [14][15] in multiple regions (for example, North America and Europe). The D-Wave quantum annealer is a programmable Ising processing unit (IPU or quantum processing unit QPU) that can find the minimum of the (stochastic) Ising Hamiltonian or its equivalent quadratic unconstrained binary optimization (QUBO) form. To solve a problem on a D-Wave quantum annealer, therefore, one needs to define an Ising Hamiltonian whose ground (minimal energy) state represents a solution for the original problem of interest. More precisely, the D-Wave quantum annealer receives coefficients of an Ising Hamiltonian (here $\mathbf{h}$ and $\mathbf{Q}$) and returns the vector $\mathbf{z}$ that minimizes the following quadratic energy function:

$$E_{\text{Ising}}(\mathbf{z}) = \sum_{i=1}^N \mathbf{h}_i \mathbf{q}_i + \sum_{i=1}^N \sum_{j=i+1}^N Q_{ij} \mathbf{q}_i \mathbf{q}_j,$$

where N denotes the number of quantum bits (qubits) and $\mathbf{q}_i \in \{-1, +1\}$. Some restrictions to the general problem are imposed by the hardware design. First, the h_i and Q_{ij} weights must match the physical connectivity structure on the chip. Qubit connectivity could be based on [16] a Chimera structure denoted C_k . Each unit cell in a Chimera structure is a complete bipartite graph $K_{4,4}$ on 8 nodes. Cells are connected in a $k \times k$ grid, with four qubits connected to horizontal neighbors and four qubits connected to vertical neighbors with internal couplers. To realize the Chimera topology in hardware, groups of qubits (number depends on cell dimension) are arranged in criss-cross fashion to form the cells. External couplers connect colinear pairs of qubits - pairs of parallel qubits in the same row or column. Chimera qubits are characterized as having nominal length 4 - each qubit is connected to 4 orthogonal qubits through internal couplers, degree 6 - each qubit is coupled to 6 different qubits. Alternatively, In Advantage Pegasus structure (the one considered in this work), qubits are "oriented" vertically or horizontally, as in the Chimera topology, but similarly aligned qubits are also shifted. For QPUs with the Pegasus topology it is conceptually useful to categorize couplers as internal, external, and odd. Internal couplers connect pairs of orthogonal (with opposite

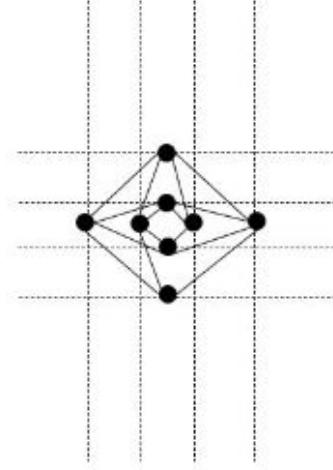


Fig. 3. Chimera cell; picture inspired by [17]

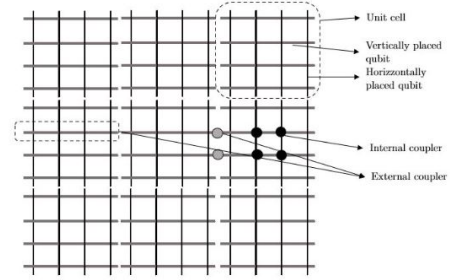


Fig. 4. Chimera grid; picture inspired by [17]

orientation) qubits. External couplers connect vertical qubits to adjacent vertical qubits and horizontal qubits to adjacent horizontal qubits. Odd couplers connect similarly aligned pairs of qubits. Pegasus features qubits of degree 15 and native K_4 and $K_{6,6}$ subgraphs. Pegasus qubits are considered to have a nominal length of 12 and degree of 15. Note that the current chip fabrication process leaves some number of qubits and couplers inoperable. Therefore each individual chip has a specific hardware working graph. After obtained a native instance is consequently necessary to map the general logical connectivity graph to the hardware working graph, Meaning to find a minor embedding of G into H . A minor of H is any graph H' that can be constructed from H by collapsing two adjacent vertices into one (combining their incident edges), or by removing an edge. Source vertex $v_i \in V_g$ can be split into multiple vertices to form a connected sub-graph t_i such that $t_i \subseteq H$. The edges of t_i form a chain that must be mapped onto H . Weights on chain edges are set to large-magnitude negative values (ferromagnetic couplings) so that all vertices in t_i have the same spin in the optimal solution. Source edge $(v_i, v_j) \in E_g$ must map to an edge $(w_i, w_j) \in H$ such that $w_i \in t_i$ and $w_j \in t_j$. weight h_i may be assigned to one vertex in t_i or split among multiple vertices. Finding appropriate embedding is crucial so Dwave Ocean API use minorminer heuristic [18] that proved to behave efficiently in terms of

performances and time to embedding. However, embedding for the graph could be specified by the user.

C. Background and Proposal

D-Wave quantum annealer architecture has limitations that restrict the process of mapping problems into an executable QMI but also lower the quality of results (not guaranteeing that executing the corresponding QMI on a quantum annealer will attain a global optimum). Adiabatic QPU have trouble excluding the impact of device imperfections and the outside environment. Noise could cause decoherence (that leads to an erroneous solution sampled) or more general lower performances of solution. These aspects are typically handled directly by the QPU also via error correction mechanisms (postprocessing techniques). Typically, noise could come from different sources [19]

- background susceptibility which comes from next-nearest neighbor interactions and leakage of biases
- flux noise is exerted on qubits, which manifests itself as a drift of their properties on a larger timescale, such as between different problem submission
- the problem Hamiltonian, which is specified as part of the input to the machine has a finite resolution. The current generation of the D-Wave QPUs uses 8–9 bits for representing coefficients. Hence, the D-Wave QPU truncates coefficients of a QMI prior to putting qubits in their superposition, which can result in the Ising model having a different ground state.
- the ratio between biases and couplings can vary for different annealing parameters.
- qubits cannot be made perfectly identical. In addition, several studies have revealed that the QPU is inherently coupled to its environment
- thermal effects
- random quantum fluctuations in the Josephson current present in qubits

It is typically a burden of the user to find adequate hyperparameters setting to carry on annealing process successfully (a fully list is available here [20]).

1) *Previous works*: There are actually three works that try to analyze or setting parameters aided by machine learning algorithms. [21] implement a S-type learning automata (a particular reinforcement learning automata). This automata views the anneal process as atomic and constructs an Ising Hamiltonian to improve results on the problem at hand (This is justified by a claim, proved in the paper itself, that assert for every NP problem there are infinite Ising models whose ground states are all identical to the solution of the original problem). The problem considered is 3-sat with penalty formulation. At each iteration automata samples from phase transition of a 3-sat instance (with maximum of 50 variables) applying penalty to every unsatisfiable variable. [19] focus on maximum clique problem via random instance with maximum graph dimension of 65 (maximum embeddable clique in annealer). They try predicting solvability of problem (classification task) and maximum clique extracted (regression task). To do that they considered decision tree classifier for classification task

and gradient boosting regressor for regression task. They choose these models for their capability of ranking features (via permutation importance ranking). To teach model they use features precomputed on minor embedding graph. They leave solver parameters (p.e. anneal time) mainly unchanged (or heuristically tuned). They optimize parameters of classifier/regressor via grid search. They leave open generalizability of this approach and if there are better model. [22] author suppose that all sources of noise could be summarized by a calibration term summed to Q matrix. So, they try to improve output of annealer via a machine learning parametrization of Q matrix that is used in Ising formulation. This is done adding a different calibration term to each of the entries of Q matrix (this is deduced from the fact that topology and position of embedding both influence results of annealing). The problem considered here is finding ground state of an electron system over triangular lattice. They find simple linear method more accurate than other methods. To train the linear model, they propose sampling random calibration mask and use result of anneal process with sampled anneal mask to find correction terms. Note also how [22] reparameterization could be seen as a variation of [21] approach given they both act on Q matrix changing couples and bias values. Some observations have to be done:

- Notice that all this papers use D-Wave architecture with chimera graph. At the moment however, the new Pegasus topology is available [23], that allow more densely connected minor graphs and consequently handle larger problem and give robustness.
- None of them use deep learning frameworks.
- There is no dataset that explicitly handle the tasks of this publications (so as it would be exposed it is necessary to obtain an appropriate dataset).
- [21] and [22] not deal with graph structure. They base their predictions only on Q and h matrix; note however that Q and h matrix are then mapped on an embedding graph on which annealing process is actually carried on

2) *Proposal*: Of all parameters in direct control of the user which affect the finding of a solution (a fully list is available here [20]) this work focuses on minor embedding. Motivations which lead to this choice are:

- being minor embed a graph, it could be handled via Graph Neural Network which are one of main focuses of the work.
- actual annealing operate on minor embedding, so its properties are among the one which directly influence performances. Methods which operates on Q and h matrix (like [21] and [22]) act de facto indirectly over this embedding

As anticipated, work focuses on Graph Neural Networks. As previously seen no previous work on analyzing or improving annealing focuses on neural networks even if neural networks offer advantages: avoid user finding precomputed features, allow for easy construction of complex model training different component as a whole via backpropagation and are known to achieve state of the arts performances on different tasks (and being often the only possibility for generative tasks). The work

is then divided in two parts

- first part is devoted to compare [19] classifier performances to an ad hoc constructed GNN in predicting quantum annealer performances on a maximum independent set task. Differently from original work, author of this document allows for greater graphs dimension and real word graphs data extending experiments of their previous work.
- second part focus is on Generative Graph Neural Network to improve minor embedding performances on lower performance instances of the first task.

Note that having at disposal a tool capable to distinguish solvable and unsolvable instances allow for

- spare QPU utilization avoid executing unsolvable instances
- find which parameters are more influential in determine solvability

At the same time improving minor embedding could allow spare QPU execution time and allow for finding new rules to better optimize embedding.

III. FIRST EXPERIMENT - PERFORMANCE PREDICTION OF QUANTUM ANNEALING TASK

In this section first experiments and its results are described. First section describe dataset construction and elaboration. Second section describe GNN model used. Third section describe experiment result.

A. Dataset Construction

Here author construct a dataset similar to the one used by [19] which contains instances of maximum clique problems subjected to quantum annealing. Maximum clique problems are typical benchmarks. In this work this approach is selected for comparison with [19]. However it should be noted that other complex problems like weighted max cut [24] are directly embeddable in Ising hamiltonian similarly to the task at hand. The maximum clique problem is equivalent to the Maximum Independent Set problem and the Vertex Cover problem. It is also used as an intermediate step in solving the Maximum Common Subgraph problem. In this work it is considered the optimization version of the problem meaning we are trying finding greatest clique possible. Given an undirected graph $G = (V, E)$, a clique is a subset S of the vertices forming a complete subgraph, meaning that any two vertices of S are connected by an edge in G . The clique size is the number of vertices in S , and the maximum clique problem is to find a clique with a maximum number of vertices in G [25]. QUBO formulation of this problem is taken from [26]: Recall that a QUBO problem can be written as

$$\text{minimize}_{x_i \in \{0,1\}} H = \sum_{1 \leq i < j \leq N} a_{ij} x_i x_j,$$

where the weights $a_{ij}, i \neq j$, are the quadratic terms and a_{ii} are the linear terms (since $x_i^2 = x_i$ for $x_i \in \{0,1\}$). There are multiple ways to formulate the MC problem as a QUBO. One of the simplest is based on the equivalence between MC

and the maximum independent set problem. An independent set S of a graph H is a set of vertices with the property that for any two vertices $v, w \in S$, v and w are not connected by an edge in H . It is easy to see that an independent set of $H = (V, \bar{E})$ defines a clique in graph $G = (V, E)$, where $\bar{E}$ is the complement of set E . Therefore, looking for the maximum clique in G is equivalent to finding the maximum independent set in H . The corresponding constraint formulation for MC is

$$\begin{aligned} & \text{maximize}_{x_i \in \{0,1\}} \sum_{i=1}^N x_i \\ & \text{subject to} \quad \sum_{(i,j) \in \bar{E}} x_i x_j = 0, \end{aligned}$$

where $G = (V, E)$ is the input graph and $\bar{E}$ is the complement of E . The equivalent unconstrained (QUBO) minimization of previous equation written in the QUBO form above is

$$H = -A \sum_{i=1}^N x_i + B \sum_{(i,j) \in \bar{E}} x_i x_j,$$

where one can determine that the coefficients/penalties A and B can be chosen as $A = 1, B = 2$ [27]. Given typically benchmark set for MC are too big to be handled by annealer (e.g. [28]) author decides to generate graphs of the problem:

- half of the instances come from Erdős–Rényi graph generated via "erdos_renyi_graph" method of networkx package [29]. Erdős–Rényi (ER) model [30] define the likelihood of an edge occurring between any pair of nodes is defined

$$P(\mathbf{A}[u, v] = 1) = r, \forall u, v \in \mathcal{V}, u \neq v,$$

where $r \in [0, 1]$ is parameter controlling the density of the graph. This approach is followed by [19][25][31]. Author used same density r (randomly 0.01 to 0.99) of [19] but extend number of edges to 150 (from 64 of [19]), empirically founded maximally possible embeddable graph on annealer.

- As already noted by other author ER has limitations in approximating real graph [32]. It is supposed that present work approach should operate also on real graph instance. Given presence of TUD dataset [33] which has proven to be a useful benchmark for graph classification (of which graph are taken from real dataset), differently from [19], author decide to employ a random sampling from TUD datasets graphs when number of node do not exceed maximum embeddable size to include also real world graph and to create a more real world graph representative dataset. Clearly, using a uniform distribution to select between which TUD dataset sample a graph, having no prior information on a supposed existent total real graph distribution has several limitation in obtaining such dataset. There exist some work in which no prior is assumed for stratigraphic sampling [34] but this suppose disjunctive class which is not necessarily the case for a graph generating distribution. TUD datasets are also not exhaustive in respect of universe of real world graph making stratified sampling impossible. Author argue this is however a good compromise to extend previous [19]. Note also how TUD dataset provides undirected cleaned

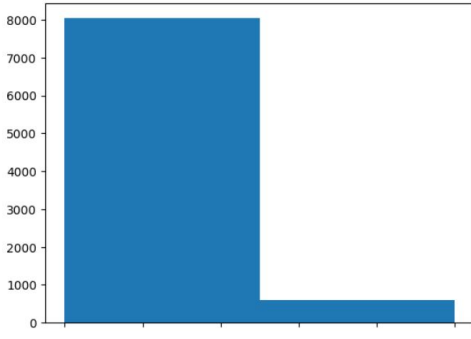


Fig. 5. Class proportions

dataset versions meaning non isomorphic data instances which allows avoid isomorphism bias as motivated by [35]

Following [19] author define an instance solvable by annealer if maximum clique founded is the same of "graph_clique_number" method of networkx package [36]. Data are collected using as embedding the one extracted by minorminer [37]. Other parameters are setted as in [19]

- anneal time randomly extracted from interval 1-1000 ms
- torque value randomly extracted from interval 0.5-3.0
- number of reads 1000

For each data item author extracted

- original Ising graph
- embedded graph
- all graph computed features of [19] for both input and embedded graph (see [19] for a complete list)

The total dataset is then composed of 8648 data instances requiring around 1 month of continual execution to be produced. Due to high imbalance in classification labels, in line with [19], (see figure 5) author decide to oversample the datasets. Other possibilities in case of imbalanced datasets are available. However note that transformations can not be used in present task given after a transformation is applied over an embed graph it is not possible to determinate if transform would lead to a solvable instances. Undersampling is not an option given the high imbalance. There are not methods for imbalance handling in graph classification apart of methods which augment graphs [38][39]. This methods are avoided for same reason of graph augmentation

B. Model description

The GNN is constructed to take as input embedding of quantum annealer:

- nodes have as attributes the actual node on the annealer architecture and the bias value on the actual embedding graph.
- edges have as attributes actual node on the annealer architectures and coupling strength on actual embedding graph.

Having no previous work at disposal that use GNN to deal with quantum annealer embedding graph author decide to resort to a method to search for GNN architecture. Authors decide to

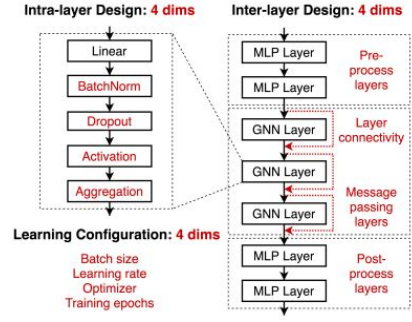


Fig. 6. Design space of graphgym (and present work). Picture taken from [40]

take inspiration from design space of [40], but instead of using ready available tool, author decides to reimplement it from scratch to

- allow for usage of convolutional layer with deal with edge features
- employ BOHB optimization [41] instead of simple grid search used by [40]

[40] and present design space is composed of interlayer design (meaning number of pre and post process linear layer and final aggregation) and intra layer design (each convolutional layer is followed by batchnorm, dropout and activation). A network of this type is then composed of a set of linear layers (preprocess layers) followed by a stack of graph convolutional layers followed by a set of linear layers (post process layers). The design space employed is then composed by

- activation = { ReLU, PReLU, Swish }
- dropout = { 0.2, 0.4, 0.6 }
- aggregation = { mean, max, add }
- jumping knowledge = { None, cat, max, LSTM }
author decide to employ jumping knowledge layers instead of skip connection given proved increased performances in several task [42][32]
- normalization = { None, batch, graph, pair, mean } extend graphgym normalization strategies including graph norm [43], pair norm [44] MeanSubtractionNorm [45]
- message passing layers = { gat, reset, conv, pdn }
author select message passing layers based on capability to handle edge features which in embed graph represents embed coupling strength and which are known to be core component of embed efficacy. These are GATConv [46], ResGatedGraphConv [47], GATv2Conv [48], PDNConv [49]. Design space also contain parameters of these convolutions and extend original [40] design space for convolution
- batch size
- scheduler = None, CosineAnnealingWarmRestarts, CyclicLR, ReduceLROnPlateau
which was not covered by [40]. Cyclic scheduling proved to be beneficial in multiple domains [50][?] while reducing learning rate when improvement is reduced is another standard technique [51].
- optimizer = { AdamW, Adamax, NAdam, RAdam }
extending [40] which consider only Adam and SGD

optimizer. Here author decide to focus on Adam variants which prove efficient in several tasks. In particular AdamW [52], Adamax [53], NAdam [54], RAdam [55]

- preprocess linear layer (from 0 to 3)
- post process linear layer (from 0 to 3)

These design space is integrated considering other hyperparameters which are typically optimized in neural networks. Hyperparameters of neural networks are typically divided in [56] regularization parameters, learning parameter, preprocessing parameters and weight initialization parameters but typically are considered worth to be optimized only nonlinearity (already present in design space), number of epochs, optimizer hyperparameters, label smoothing and drop out rate (already present in previous described design space) [57][58]. So author insert also the following hyperparameters in BOHB optimization:

- batch size: default 32 (a minimum of 16 given 10 is the minimum for employ gpu)
- weight decay: [59] suggest between 10-5 to 10-3 or 0 but author finds a value of 0.1 still worth to be considered
- label smoothing: [60] a default value of 0.1 is suggested so it was also tested if label smoothing improve results
- optimizer parameters
- learning rate: for optimize learning rate authors decide to bound possible learning rate values used in search using a learning rate range test [61]

Design space is then optimized using BOHB optimization [41] which is considered state of the arts neural network parameter optimization approach [62].

C. Results

Author compared GNN optimized neural networks over two subtask.

- first subtask concern predict if annealer is capable to solve an instance
- second subtask concern predict number of vertex of the clique the annealer have found (0 if annealer cannot find any clique), meaning a regression task.

For both experiments also [19] experiments were repeated as following described to compare results of authors proposed GNNs. Comparison were done using stratified k-fold crossvalidation over 5 splits, evaluating significance of results using deep significance framework [63], in particular using ASO scores (ASO does not make any assumptions about the distributions of the scores and could be applied to any type of metric [64])

1) *Instance solving prediction*: Crossentropy loss is used for training neural network given high confidence prediction and stability. F1 scores are used for model comparisons which summarizing precision and recall are adequate for the task of predicting labels which are very disproportionate in their distribution. Following [19] authors trained a tree classifier after doing a stratified grid search over parameters max depth (from 2 to 10) and minimum impurity decrease (0.0005 to 0.5). Results can be seen in figure 7. Even if ASO do not allow to state a dominance of a strategy on the another one, neural

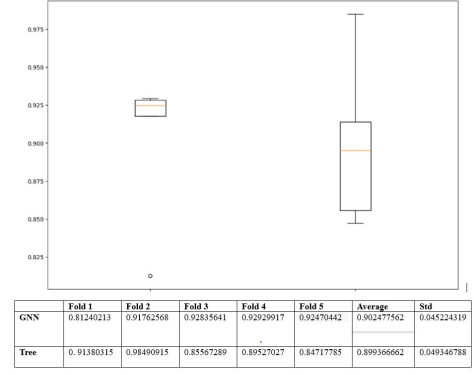


Fig. 7. Performances for classification task

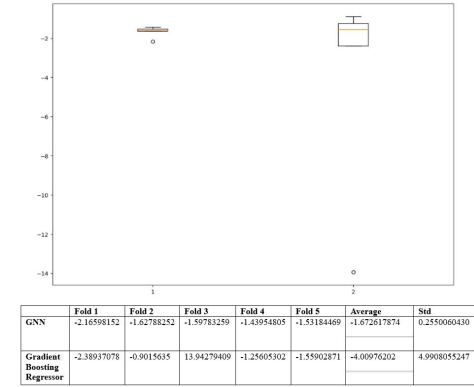


Fig. 8. Performances for regression task

network prediction demonstrate to have lower variability and being more stable while achieving better average results.

2) *Number of clique regression*: smooth L1 loss is used given it is less sensible to outliers than MSE loss. Here comparisons of results is done with average negative MSE as done in [19]. Following [19] authors trained a gradient boosting regressor after doing a stratified grid search over parameters max depth (same range as before), minimum impurity decrease (same range as before) and number of regressors (100 to 500). Results can be seen in figure 8. As in previous experiment even if ASO do not allow to state a dominance of a strategy on the another one, neural network prediction demonstrate to have lower variability and being more stable while achieving better average results.

IV. SECOND EXPERIMENT - PERFORMANCE IMPROVEMENT OF QUANTUM ANNEALING MINOR EMBEDDING

Second experiment involve creation of a method to improve embedding generation to solve instances of maximum clique (optimization version problem). First subsection regard construction of the datasets employed during this experiment, second subsection regard a description of the model while third section describe actual results.

A. Dataset Construction

For the improvement of quantum annealing embedding, a subset of data of the first experiments was extracted satisfying two properties:

- instance was not solved and no clique was founded by annealer
- number of nodes is lower than 100

First criteria allow to select hardest instances for minorminer while second instance allow reducing annealing time (note in fact that actual D-Wave platform allow to a maximum of 1 minute usage for account) while still be comparable with maximum number of variables of the embedding that was previously optimized [21] (note however that this number of variable was not a technical limitation of the approach. Further studies with more resources could be done in principle with a greater number of variables).

Another dataset was also produced to pretrain the module in charge of producing adversarial reward (see next section). This dataset should allow a classifier to discriminate between an embedding instances and another embedding instances which could or could not represent the same QUBO/Ising problem. In fact, for any problem in class NP, there are infinite different Ising models (and consequently QUBO problems) whose ground states are all identical to the solution of the original problem [21]. This is fast demonstrated by the fact that according to Cook—Levin theorem, one can reduce any NP problem to finding the ground state of Hamiltonians (which is also in the class NP) in polynomial-time. Multiplying all coefficients of the Ising model by a positive non-zero real number will result in a new spin glass problem whose ground state will be identical to the original Ising model [21]. From the point of view of solving a problem via a quantum annealer also including a node connected to another one do not alter the embedding as long as not cause a chain break. [65] suggest that a rule of thumb is that a chain break is avoided as long as chain strength is higher than the QUBO entries. Consequently this second dataset is generated taking an instance from the first exercise dataset and adding random operations which could be characterized as

- adding a node
- adding an edge with a selected strength
- multiply an edge strength by a real value

This second dataset is consequently generated iterating over 1000 instances of first exercise dataset and applying the aforementioned operations n times (in present case 10 consequent operations for each graph). For each of the original problem instances 5 different variations were produced, on two of them enforcing that they represents same problem instance.

B. Model description

The task at hand poses unique constraints of possible approaches that could be exploited to generate an appropriate embedding

- D-Wave embedding graph have hard constraints posed by the presence of a specific architecture with only a subset of nodes available at any time and only a subset of edges.

- An embedding of a Ising/QUBO instance should associate a subset of embedding graph to a subset of variables of the original problem
- A generated embedding graph should have a variable number of nodes and edges
- It is appealing to start by a minorminer or user defined embedding and improve on it also to exploit previous knowledge of the problem

The only deep graph generation approach which can deal with hard constraints and variable number of nodes is [66] so author decides to take inspiration on it. Model proposed then take as input a minor graph and employ a finite number of discrete operations to create a novel embedding. More formally description is now given.

Define a graph G as (A, E, F)

- $A \in \{0, 1\}^{n \times n}$ is the adjacency matrix
- $F \in R^{n \times d}$ is the node feature matrix assuming each node has d features (two in our case)

Our primary objective is to generate graphs that maximize a given property function $S(G) \in R$, i.e., maximize $E_{G'}[S(G')]$, where G' is the generated graph, and S could be one or multiple domain-specific statistics of interest: in present case statistics of interest is dimension of clique founded by annealing process. The procedure to generate a graph can then be described by a trajectory $(s_0, a_0, r_0, \dots, s_n, a_n, r_n)$, where s_n is the final generated graph. This can be formulated as a general decision process $M = (S, \mathcal{A}, P, R, \gamma)$ where

- $S = \{s_i\}$ is the set of states that consists of all possible intermediate and final graphs. s_t is the intermediate generated graph G_t at time step t , which is fully observable by the RL agent. G_0 is the starting graph, meaning minorminer embedding (original [66] work used instead a single node that represents a carbon atom).
- $\mathcal{A} = \{a_i\}$ is the set of actions that describe the modification made to current graph at each time step. Let's define a set of scaffold subgraphs $\{C_1, \dots, C_s\}$ to be added during graph generation and the collection is defined as $C = \bigcup_{i=1}^s C_i$. Given a graph G_t at step t , define the corresponding extended graph as $G_t \cup C$ which is the input to the agent. Under this definition, an action can either connecting a new subgraph C_i to a node in G_t (as long there is an edge in the embedding), disconnecting two nodes in G_t (as long disconnecting two nodes does not make a chain disconnected or do not disconnect two chains which should be connected to represent two interacting variable in the original QUBO) terminate graph generation. Once an action is taken, the remaining disconnected scaffold subgraphs are removed to evaluate the generated graph. As in original work author adopts the most finegrained version where C consists of all b different single node graphs, b equal number of available node in real architecture not involved in other chains.
- P is the transition dynamics that specifies the possible outcomes of carrying out an action. The state transition distribution: $p(s_{t+1} | s_t, \dots, s_0, a_t) = \sum_{a_t} p(a_t | s_t, \dots, s_0) p(s_{t+1} | s_t, \dots, s_0, a_t)$, where $p(a_t | s_t, \dots, s_0)$ is represented as a policy network π_θ (which requires

the state transition dynamics to satisfy the Markov property: $p(s_{t+1} | s_t, \dots, s_0) = p(s_{t+1} | s_t)$. Domain-specific rules are incorporated in the state transition dynamics. Infeasible actions are rejecting with a negative reward

- $R(s_t)$ is a reward function that specifies the reward after reaching state s_t , and γ is the discount factor. Rewards are given differently intermediately and finally. Intermediate rewards are sum of stepwise validity rewards and adversarial rewards. Validity reward is given if a classifier network predict that embed graph solve the problem. Final rewards are a sum over domain-specific rewards (given if the annealer actual find a solution with final embedding produced, this is also used to train the discriminator used for intermediate rewards if it would have predicted an erroneous result) and adversarial rewards. Adversarial reward $-V(\pi_\theta, D_\phi)$ is included to enforce soft constraints that define belonging of the graph to the class of possible original problem embedding (originally in [66] was a class of molecule) and employ the Generative Adversarial Network (GAN) to define it as:

$$\min_{\theta} \max_{\phi} V(\pi_\theta, D_\phi) = E_{x \sim p_{\text{data}}} [\log D_\phi(x)] + E_{x \sim \pi_\theta} [\log D_\phi(1-x)]$$

where π_θ is the policy network, D_ϕ is the discriminator network, x represents an input graph, p_{data} is the underlying data distribution which defined either over final graphs (for final rewards) or intermediate graphs (for intermediate rewards). Note that only D_ϕ can be trained with stochastic gradient descent, as x is a graph object that is non-differentiable with respect to parameters ϕ (in any case adversarial discriminator is trained every time is called for give an adversarial reward).

To train the network authors adopt Proximal Policy Optimization (PPO) as in [66][67], which was implemented from scratch. The objective function of PPO is defined as follows

$$\max_{\theta} L^{\text{CLIP}}(\theta) = E_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

where

- $r_t(\theta)$ is the probability ratio that is clipped to the range of $[1 - \epsilon, 1 + \epsilon]$,

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

- $\hat{A}_t$ is the estimated advantage function which involves a learned value function $V_\omega(\cdot)$ to reduce the variance of estimation. In present work, $V_\omega(\cdot)$ is a network identical to the actor network.

whose parameters were tuned manually to achieve the resulting performances.

To model actor and critic network authors resort to a "two-tail" model meaning the neural network is composed by

- first tail: a module composed by a set of linear layers and a stack of graph convolutional neural networks (with a

design identical to best configuration for the classifier networks founded in previous experiments) which take as input the graph representation of the original QUBO problem

- second tail: an analogous model composed which take as input current and scaffolds graph
- an head which take as input features computed by the two tails and produce the appropriate action

The action a_t at time step t is a concatenation of three components: selection of two nodes and prediction of action type (add, remove or terminate):

$$a_t = \text{CONCAT}(a_{\text{first}}, a_{\text{second}}, a_{\text{operation}})$$

where

- $a_{\text{operation}}$ is obtained applying a softmax over the output of a linear layer which maps representation computed by concatenation of features of first and second tail as done in [66].
- a_{first} is obtained applying a softmax over the output of a linear layer which maps representation computed by second tail for every node that is actually involved in the embedding as done in [66]. Note that each node involved in embedding is part of a chain which represents node of the graph of the original QUBO/Ising problem. The node representation of each node involved in the actual embedding is then concatenated to the embedding of the node of the original QUBO graph (obtained by the first tail) it represents in the chain in which it is involved (before feeding to linear layer).
- a_{second} depends if the action is an add or delete edge action. The second case is obtained as for the first node but concatenating to each node embedding the node embedding of the first selected node (as done in [66]). In the case of a delete action the output is instead related to any of the scaffold node meaning node not contained in the actual embedding. Also in this case the node embedding of the first selected node are concatenated to the scaffold node embedding

As said before environment employ

- a classifier network (which used pretrained weights by previous experiment) that assign a positive reward if the actor network produce an embedding which is classified as solving the problem. This network is trained via backpropagation (optimizer and optimizer parameter are the same which were found to be the best in previous experiment) everytime the environment produce the final reward (actually enquiring the annealer). Note how this allow not always enquire dwave annealer every step, avoid wasting QPU usage.
- an adversarial network which is in charge of producing adversarial reward and it is trained via backpropagation every time is asked to produce adversarial reward.

The adversarial network is composed by two graph autoencoder [68] and an head. Each of the two autoencoder are fed with a separate graph. Their encoded representation is concatenated and fed to the head which discriminate if the two graph represents same original QUBO/Ising problem instance

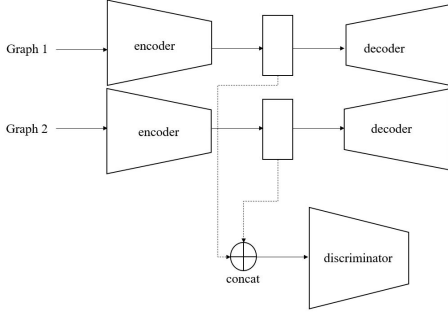


Fig. 9. Adversarial reward neural network component

or not. The adversarial reward is obtained then during PPO summing two components

- $E_{x \sim p_{\text{data}}} [\log D_{\phi}(x)]$ take as input the starting embed graph for the current problem instance and the same graph modified with an operation that allow obtain a graph representing same problem instance (see dataset section before)
- $E_{x \sim \pi_{\theta}} [\log D_{\phi}(1 - x)]$ take as input the starting embed graph for the current problem instance and the actual generated graph

The architecture is depicted in figure 9. Given importance of pretraining for graph neural network [69], this module is pretrained on a dataset of graph instances obtained by first experiments. During this task is porpouse is classifying couple of graphs which represents same original QUBO/Ising instances. This task employ a composite loss that is composed by the sum of autoencoder reconstruction loss, crossentropy loss for classification task and cosine embedding loss to enforce separation or closeness of the autoencoder embedding if the graph represent respectively different or same instance.

C. Results

The whole architecture is then trained for 50 epochs. This is a limited number however restricted by the extremely limited available usage for a token API key (1 minute of QPU usage) which requires to change a key every PPO iteration. Performances are evaluated as in [66] considering average improvements over dimension of clique founded by embedding discovered by the actor. To have more robust results (given crossvalidation could not be applied) author decides to report results over 5 independent test sets extracted from the data. Each sets contains 62 graphs. Results are reported in figure 10. From the results it is apparent that the policy founded is capable of increasing dimension of clique founded over particular instances (high standard deviation). Even if this result is encouraging it would be better having a more stable policy. More experiments should be conducted to improve stability of the policy learned. This has probably to resort to better rewards and more recent proximal policy optimization algorithms. Having at disposal a greater access to D-Wave platform it would be also of capital importance to resort to more sistematic parameter tuning.

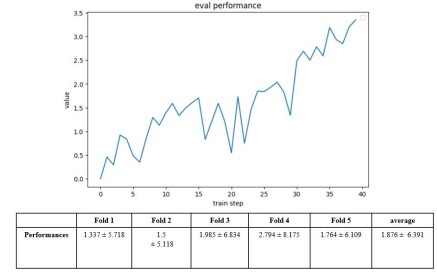


Fig. 10. Results of reinforcement learning based architecture

V. CONCLUSION

A. Results

A dataset of around 9000 instances which requires around 1 month to be built. Results of [19] where extended to different (TUD dataset) and high dimensional graph confirming their results about capability to predict embedding performances while at the same time showing how a graph neural network could obtain comparable performances in the same task with low prediction variance. A new generative model capable of improving embedding was proposed. This model is the first to be able to act to a predefined (user or minorminer) given embedding and can in principle be applied to general problem instances. While showing unstability in graph improvement author argues could represent a valuable starting point for future improvements.

B. Possible Improvements

More extensive experiments should be conducted on other problem instances which are typcally used as benchmarks. As an example SAT problem could be solved as instance of maximum independent set problem. However conducting these experiments would need extensive computation on QPU which would require more time available for an API token. Same limitations applies to PPO reinforcement learning agents (as underlined in second experiments results section). Original [19] work employed decision tree as offering generally more explainability (permutation ranking of features). New methods of AI explainability could be applied to graph neural network in both experiments to obtain information regarding which features networks exploit to classify solvable instances or producing a policy.

REFERENCES

- [1] C. Schuman, T. Potok, P. Date, and R. Patton, "A classical-quantum hybrid approach for unsuper-vised probabilistic machine learning." *FICC 2019: Advances in Information and Communication*, 2020.
- [2] L. Kish, "End of moore's law: thermal (noise) death of integration in micro and nano electronics," *Physical Letters A*, 2002.
- [3] R. Perez-Castillo, J. Hevia, M. Serrano, G. Hernandez, I. de Guzman, C. P. Polo, M. Murina, M. Piattini, and G. Peterssen, "The talavera manifesto for quantum software engineering and promgramming," 2020.
- [4] R. Babbush, D. Bacon, J. Bardin, R. Barends, R. Biswas, S. Boixo, F. Brandao, A. Buell, B. Burkett, Y. Chen, Z. Chen, B. Chiaro, R. Collins, W. Courtney, A. Dunsworth, E. Farhi, B. Foxen, A. Fowler, C. Gidney, M. Giustina, R. Graff, K. G. K., J. Martinis, F. A. F., and K. Arya, "Quantum supremacy using a proqrammable superconducting processor," *Nature*, 2019.

- [5] S. Gutmann, J. Lapan, A. Lundgren, D. Preda, E. Farhi, and J. Goldstone, "A quantum adiabatic evolution algorithm applied to random instances of an np-complete problem," *Science*, 2001.
- [6] S. Gutmann, M. Sipser, E. Farhi, and J. Goldstone, "Quantum computation by adiabatic evolution," *ArXiv*, 2000.
- [7] Z. Landau, S. Lloyd, O. Regev, D. Aharonov, W. van Dam, and J. Kempe, "Adiabatic quantum computation is equivalent to standard quantum computation," *SIAM Journal of Computing*, 2007.
- [8] "D-wave home," <https://www.dwavesys.com/>.
- [9] A. Ruiz, "Quantum annealing, brief description on the state of the art of some local optimization methods," 2014.
- [10] A. L. Ny, "Introduction to (generalized) gibbs measures," *ensaios matematico*, 2008.
- [11] F. Petruccione, M. Schuld, and I. Sinayskiy, "The quest for a quantum neural network," *Quantum Information Processing*, 2014.
- [12] "Let's get practical*: D-wave details product expansion cross platform roadmap," <https://www.dwavesys.com/company/newsroom/press-release/let-s-get-practical-d-wave-details-product-expansion-cross-platform-roadmap>.
- [13] "product overview: The only end-to-end quantum platform," <https://www.dwavesys.com/solutions-and-products/product-overview/>.
- [14] "The leap the only real-time quantum cloud service built for business," <https://www.dwavesys.com/solutions-and-products/cloud-platform/>.
- [15] "solver docs," https://docs.dwavesys.com/docs/latest/guides_solvers.html#guides-solvers.
- [16] "D-wave documentation: D-wave qpu architecture: Topologies," https://docs.dwavesys.com/docs/latest/c_gs_4.html#topology-intro-pegasus.
- [17] C. McGeoch, *Adiabatic Quantum Computation and Quantum Annealing, Theory and Practice*, first edition ed. MorganClaypool publishes, 2014.
- [18] J. Cai, G. Macready, and A. Roy, "A practical heuristic for finding graph minors," *ArXiv*, 2014.
- [19] A. Barbosa, E. Pelofske, G. Hahn, and H. Djidjev, "Using machine learning for quantum annealing accuracy prediction," *ArXiv*, 2021.
- [20] "Solver parameters," https://docs.dwavesys.com/docs/latest/c_solver_parameters.html.
- [21] R. Ayanzadeh, M. Halem, and T. Finin, "Reinforcement quantum annealing: A hybrid quantum learning automata," *Scientific Reports*, vol. 10, 2020.
- [22] J. Brencse, D. M. V. Kabanov, L. Todorovski, S. D. Zeroski, and J. Vodeb, "Boosting the performance of quantum annealers using machine learning," *Mdpi Algorithms*, vol. 14(6), 2021.
- [23] "D-wave architectures," https://docs.dwavesys.com/docs/latest/c_gs_4.html.
- [24] P. Hegde, G. Passarelli, G. Cantele, and P. Lucignano, "Deep learning optimal quantum annealing schedules for random ising models," *New Journal of Physics*, 2023.
- [25] E. Balas and C. Yu, "Finding a maximum clique in an arbitrary graph," *SIAM J Comp*, 1986.
- [26] G. Chapuis, H. Djidjev, G. Hahn, and G. Rizk, "Finding maximum cliques on the d-wave quantum annealer," *ArXiv*, 2014.
- [27] A. Lucas, "Ising formulations of many np problems," *Frontiers in Physics*, 2014.
- [28] "Dimacs 10 erdos renyi graph," <https://sites.cc.gatech.edu/dimacs10/archive/er.shtml>.
- [29] "erdos renyi graph," https://networkx.org/documentation/stable/reference/generated/networkx.generators.random_graphs.erdos_renyi_graph.html.
- [30] P. Erdős and A. Rényi, "On the evolution of random graphs," *Publ. Math. Inst. Hung. Acad. Sci*, 1960.
- [31] E. Pelofske, G. Hahn, and H. Djidjev, "Solving larger maximum clique problems using parallel quantum annealing," *ArXiv*, 2022.
- [32] W. Hamilton, *Graph Representation Learning Book*. McGill University.
- [33] "Tud datasets," <https://chrsmrrs.github.io/datasets/>.
- [34] S. Mohammad and E. Dougherty, "Effect of separate sampling on classification accuracy," *Bioinformatics*, 2014.
- [35] S. Ivanov, S. Sviridov, and E. Burnaev, "Understanding isomorphism bias in graph data sets," *ArXiv*, 2019.
- [36] "graph clique number," https://networkx.org/documentation/networkx-2.3/reference/algorithms/generated/networkx.algorithms.clique.graph_clique_number.html#:~:text=The%20clique%20number%20of%20a,itsself%20a%20list%20of%20nodes.
- [37] "Embedding composite," <https://docs.ocean.dwavesys.com/projects/system/en/stable/reference/composites.html#embedding>.
- [38] Y. Wang, Y. Zhao, N. Shah, and T. Derr, "Imbalanced graph classification via graph-of-graph neural networks," *CIKM '22: Proceedings of the 31st ACM International Conference on Information Knowledge Management*, 2022.
- [39] Z. Liu, Q. Mao, C. Liu, Y. Fang, and J. Sun, "On size-oriented long-tailed graph classification of graph neural networks," *WWW '22: Proceedings of the ACM Web Conference*, 2022.
- [40] J. You, R. Ying, and J. Leskovec, "Design space for graph neural networks," *ArXiv*, 2020.
- [41] "Smac3," <https://github.com/automl/SMAC3>.
- [42] K. Xu, C. Li, Y. Tian, T. Sonobe, K. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," *ArXiv*, 2018.
- [43] T. Cai, S. Luo, K. Xu, D. He, T. Liu, and L. Wang, "Graphnorm: A principled approach to accelerating graph neural network training," *ArXiv*, 2021.
- [44] L. Zhao and L. Akoglu, "Pairnorm: Tackling oversmoothing in gnns," *ArXiv*, 2019.
- [45] C. Yang, R. Wang, S. Yao, S. Liu, and T. Abdelzaher, "Revisiting oversmoothing in deep gnns," *ArXiv*, 2020.
- [46] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," *ArXiv*, 2018.
- [47] X. Bresson and T. Laurent, "Residual gated graph convnets," *ArXiv*, 2022.
- [48] S. Broody, U. Alon, and E. Yahav, "How attentive are graph attention networks," *ArXiv*, 2022.
- [49] B. Rozemberczki, P. Englert, A. Kapoor, M. Blais, and B. Perozzi, "Pathfinder discovery networks for neural message passing," *ArXiv*, 2021.
- [50] L. Smith and N. Topin, "Exploring loss function with cyclical learning rates," *ArXiv*, 2017.
- [51] Y. Bengio, "Practical recommendations for gradient-based training of deep architectures," *Neural Networks: Tricks of the Trade*, 2012.
- [52] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *ArXiv*, 2017.
- [53] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," *ArXiv*, 2014.
- [54] T. Dozat, "Incorporating nesterov momentum into adam," *Proceedings of the 4th International Conference on Learning Representations*, 2016.
- [55] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han, "On the variance of the adaptive learning rate and beyond," *ArXiv*, 2020.
- [56] F. Hutter, J. Lucke, and L. Schmidt-Thieme, "Beyond manual tuning of hyperparameters," *KI - Künstliche Intelligenz*, 2015.
- [57] M. Claesen and B. D. Moor, "Hyperparameter search in machine learning," *ArXiv*, 2015.
- [58] L. Yang and A. Shami, "On hyperparameter optimization of machine learning algorithms: Theory and practice," *Neurocomputing*, 2020.
- [59] L. Smith, "A disciplined approach to neural network hyper-parameters: part 1 – learning rate, batch size, momentum and weight decay," *ArXiv*, 2018.
- [60] C. Szegedy, V. Vanhoucke, S. Ioffe, and J. Shlens, "Rethinking the inception architecture for computer vision," *Conference: Computer Vision and Pattern Recognition 2016*, 2015.
- [61] L. Smith, "Cyclical learning rates for training neural networks," *ArXiv*, 2017.
- [62] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated Machine Learning methods, systems, challenges*. springer.
- [63] "deep significance," <https://deep-significance.readthedocs.io/en/latest/>.
- [64] R. Dror, S. Shlomov, and R. Reichart, "Deep dominance - how to properly compare deep neural models," *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [65] "Setting the chain strength," https://www.dwavesys.com/media/vsufwv1d/14-1041a-a_setting_the_chain_strength.pdf.
- [66] J. You, B. Liu, R. Ying, V. Pande, and J. Leskovec, "Graph convolutional policy network for goal-directed molecular graph generation," *ArXiv*, 2019.
- [67] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *ArXiv*, 2017.
- [68] T. N. Kipf and M. Welling, "Variational graph auto-encoders," *ArXiv*, 2016.
- [69] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for pre-training graph neural networks," *ArXiv*, 2019.